

The Eclipse Layout Kernel

Sören Domrös¹[0000–0002–8011–8484],
Reinhard von Hanxleden¹[0000–0001–5691–1215], Miro Spönnemann², Ulf Rüegg¹,
and Christoph Daniel Schulze¹

¹ Kiel University {sdo, rvh}@informatik.uni-kiel.de, uruorumail@gmail.com,
cd.schulze@gmx.net

² Typefox GmbH miro.spoenemann@typefox.io

Abstract. The Eclipse Layout Kernel (ELK) is a collection of graph drawing algorithms that supports compound graph layout and ports as explicit anchor points of edges. It is available as open-source library under an EPL license. Since its beginning, ELK has served both as a research vehicle for graph drawing algorithms, and as a practical tool for solving real-world problems. ELK and its transpiled JavaScript cousin elkjs are now included in numerous academic and commercial projects. Most of the algorithms realized in ELK are described in a series of publications. In this paper, the technical description concentrates on the key features of the flag-ship algorithm ELK Layered, the algorithm architecture, and usage. However, the main purpose of this paper is to give the broader view that is typically left unpublished. Specifically, we review its history, give a brief overview of technical papers, discuss lessons learned over the past fifteen years, and present example usages. Finally, we reflect on potential threats to open-source graph drawing libraries.

Keywords: Automatic Layout · Layered Layout · Layout Library

1 Introduction

Freely available, high-quality graph drawing libraries are useful both for stress-testing their underlying algorithms, and for harnessing the power of automatic graph drawing in practical applications. In 2004, Jünger and Mutzel published a volume of papers on various graph drawing libraries [29]. This paper presents the Eclipse Layout Kernel (ELK), which has been developed since 2008 and is still actively maintained, with a growing user base.

The original driver of the ELK is modeling pragmatics [26], which, in a nutshell, aims to increase the productivity of designers working with graphical modeling languages. This is still the main focus of ELK, and explains why modeling languages such as SCCharts [24], Ptolemy [37], or, more recently, Lingua Franca [32], feature prominently in publications on ELK. In fact, when getting started on modeling pragmatics with the Kiel Integrated Environment for Layout (KIEL) tool [36] for model-driven engineering of statecharts [27], the developers started with the off-the-shelf layout library Graphviz [21]. It was only when they noticed that existing libraries did not fully address their needs that they

started developing and publishing their own algorithms. We believe this practical motivation, and continuously tight interaction between researchers and practitioners of model-driven engineering, has contributed to the quality and usability of ELK.

Fig. 1 displays ELK’s development timeline with selected student theses¹. The diploma thesis of Spönemann led to the first publication of the KIELER Layout of Dataflow Diagrams (KLoDD) algorithm to visualize data-flow [60]. That thesis subsequently led to the development of the ELK predecessor KIELER Layout (K Lay), which already included a predecessor of ELK’s flagship algorithm, which was at that time called K Lay Layered [53]. Each transformation step from KLoDD to ELK as well as each ELK release marks not only new algorithms and features but also big refactoring efforts, which enabled the longevity of ELK.

Years of work and several publications formed ELK as it is presented today: An open-source layout library for layered layouts, rectangle packing, tree drawing, force and stress layouts, radial layouts, and packing of disconnected components, which transpiles into the popular (see Fig. 2) npm-package elkjs², and includes bridges to Graphviz [21] and libavoid [65].

In the following, we will continue with related work. Sec. 2 presents the users of ELK and elkjs. ELK’s layout algorithms are presented in Sec. 3 with special focus on ELK Layered. We contribute an in depth discussion of ELK’s architecture, the algorithm architecture of the ELK Layered algorithm, and lessons learned from developing and maintaining a large open-source layout algorithm library in an academic context in Sec. 4. Sec. 5 presents real world examples layouted with ELK. Sec. 6 concludes the paper and presents future work on ELK.

Related Work There exist several layout libraries, graphical editors, or algorithms that utilize automatic layout, as presented by Jünger and Mutzel [29].

Dunnart [10] is a constraint based diagram editor, which utilizes constraints to create desirable layouts using libavoid [65], OGDF [2], or cola [9].

Cytoscape [15] is a layout library originally used for bio-informatics, which provides several algorithms for various use cases including webcola³, dagre, and elkjs. Similar to ELK, this allows using the same interface for various use cases.

The yWorks GmbH and their commercial yFiles tool [64] provide layout algorithms and visualizations. However, it is not open-source and the free yEd tool does not expose an API only for layout.

Zink et al. [66] support layered layouts with port constraint groups. Instead of just fixed positions or orderings, they allow grouping ports, which again may contain additional port groups that can have distinct layout constraints. In terms of ports constraints this Sugiyama algorithm is superior. However, it is especially developed for layout of hardware components and lacks ELK’s configurability.

¹ <https://www.rtsys.informatik.uni-kiel.de/en/teaching/theses/completed-theses>

² <https://github.com/kieler/elkjs>

³ <https://ialab.it.monash.edu/webcola/>

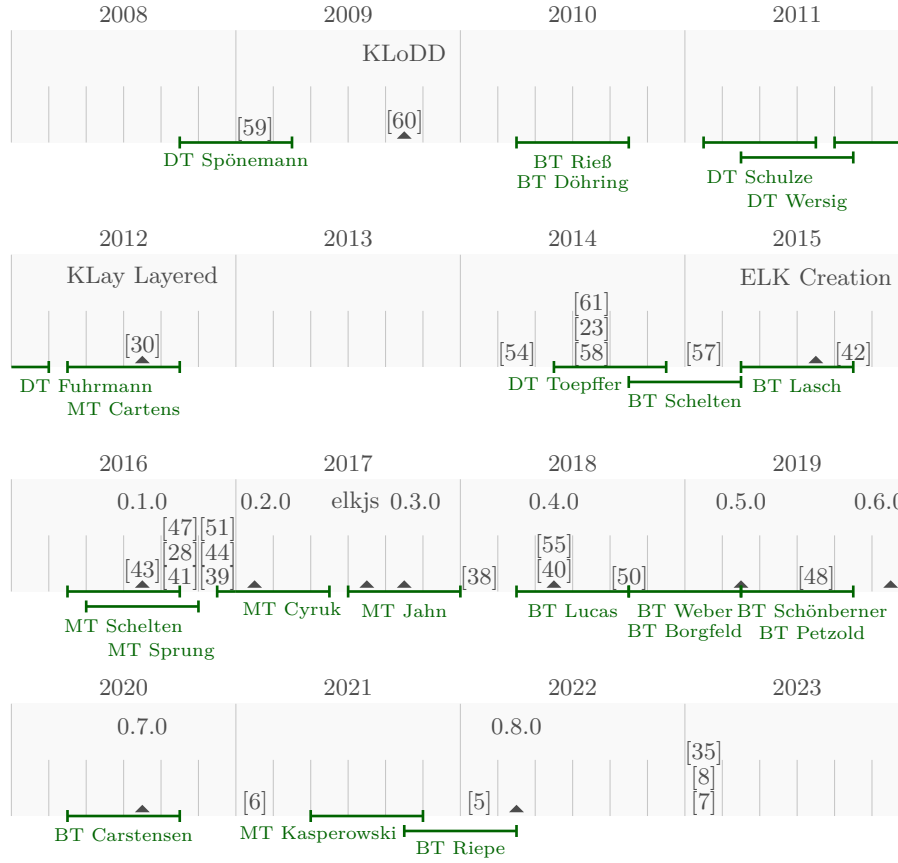


Figure 1: The ELK development timeline. Selected diploma theses (DT), bachelor theses (BT), and master theses (MT) are marked in green, software releases are marked with a gray triangle, and relevant publications are cited.

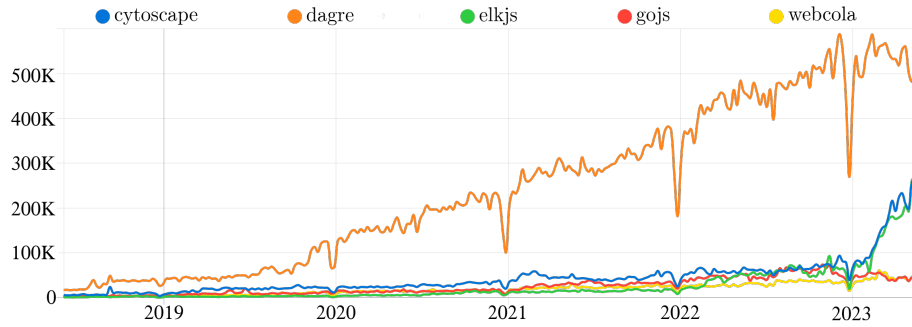


Figure 2: Weekly downloads of elkjs (green) compared to other popular layout libraries tagged with layout, graph, or dataflow on npm (<https://npmtrends.com/cytoscape-vs-dagre-vs-elkjs-vs-gojs-vs-webcola>, retrieved: 05/06/2023).

2 ELK Availability, Usage and Algorithms

ELK mainly serves as a sandbox for developing new algorithms or layout strategies, which are often used in the context of Model-Driven Engineering (MDE) to create data-flow or control-flow diagrams [26]. Most publications are based on the layered approach [5, 8, 18, 23, 25, 28, 30, 39, 40, 42–44, 53, 58, 60, 61]. The only other algorithm with a peer reviewed paper is ELK Rectangle Packing [6, 7], which is used for the layout of SCCharts [24], as seen in Fig. 7.

Availability ELK is available on GitHub⁴ and is released via maven central under EPL-2.0, which is a weak copyleft license (except ELK libavoid, which is available under LGPL, a strong copyleft license). The JavaScript library elkjs is also available on GitHub and published via npm⁵. Further information regarding ELK can be found on its Eclipse Project website⁶, the Eclipse ELK website⁷, and the elklive website⁸, which hosts a simple ELK graph editor that allows users to switch between different ELK versions, can convert between different ELK graph formats, and houses examples graphs.

Usage Since ELK is open-source and added to several other libraries and tools, we can only partly trace its usage on the basis of questions, pull-request, and issues. Notable visualization libraries that use ELK or elkjs are KLightD [46], Eclipse Sprotty⁹, Terrastruct¹⁰, Cytoscape, Mermaid¹¹, and reaflow¹². The integration of elkjs into other tools might be the reason for its increasing weekly downloads since the Cytoscape and elkjs downloads seem to correlate (see Fig. 2). These libraries make ELK available in tools such as Eclipse 4diac¹³, Eclipse Sirius¹⁴, Eclipse Papyrus¹⁵, plantUML¹⁶, the GLSP¹⁷, bigER [22], ExplorViz [13], the Kiel Integrated Environment for Layout Eclipse Rich Client (KIELER) [24], Lingua Franca [32], Capella Diagrams by Digitale Schiene Deutschland¹⁸, Ptolemy II [37], ETAS EHandbook [16], and probably many more. Since ELK is published as an Eclipse project, industrial users such as ETAS, which use

⁴ <https://github.com/eclipse/elk>

⁵ <https://www.npmjs.com/package/elkjs>

⁶ <https://projects.eclipse.org/projects/modeling.elk>

⁷ <https://www.eclipse.org/elk/>

⁸ <https://rtsys.informatik.uni-kiel.de/elklive/index.html>

⁹ <https://sprotty.org>

¹⁰ <https://terrastruct.com/>

¹¹ <https://mermaid.js.org/>

¹² <https://github.com/reaviz/reaflow>

¹³ <https://www.eclipse.org/4diac/>

¹⁴ <https://www.eclipse.org/sirius/>

¹⁵ <https://www.eclipse.org/papyrus/>

¹⁶ <https://plantuml.com/>

¹⁷ <https://www.eclipse.org/glsp/>

¹⁸ <https://github.com/DSD-DBS/capellambse-context-diagrams>

ELK nearly since its creation, can safely rely on ELK without fearing issues regarding intellectual property, since every ELK committer has to sign the Eclipse Contributor Agreement¹⁹. This enables ELK users that reach from model-driven engineers to doctors that look at medication plans layouted with ELK to use the library.

Algorithms All these tools mainly use layered layouts (ELK Layered). However, ELK also supports rectangle packing (ELK Box, ELK Rectangle Packing), tree drawing (ELK Mr. Tree), force and stress layout (ELK Force, ELK Stress), disconnected component packing (ELK DisCo), radial layout (ELK Radial), and topology aware compaction and overlap removal (ELK SpOre Compaction/Overlap Removal). Moreover, it integrates the Graphviz layout algorithms Circo, Dot, FDP, Neato, and Twopi and a bridge to libavoid [65], which does standalone edge routing. Previous versions of ELK also included a bridge to OGDF [2].

Each layout algorithm might be fairly complex. E. g. ELK Layered supports in its current state 140 layout options to further configure the Sugiyama algorithm to add spacing and configurations for edges, nodes, comments [49], labels [52], disconnected components [48], and ports [53]. Moreover, these layout options can configure and enable different layout strategies such as model order [8], node [35] and port constraints [53], node size constraints, compaction [42], edge wrapping [40], edge and node label placement [51], self-loop arrangement, top-down layout, and layout direction.

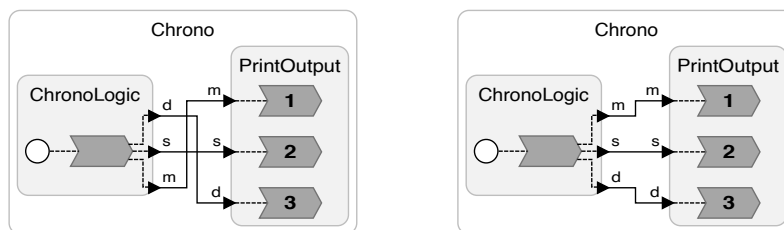
Additionally to the works mentioned in Fig. 1, ELK implements the following strategies based on works of several researchers: stress-majorization layout based on Gansner et al. [19], Fruchterman and Reingold force layout [17], Eades force layout [11], and tree node placement based on Walker [63]. Furthermore, the following algorithms are adopted for ELK Layered: a greedy cycle breaking algorithm inspired by Eades et al. and di Battista et al. [4, 12], a depth-first cycle breaker by Gansner et al. [20], a node promotion algorithm used as layer assignment post-processing, a minimal width layerer, a stretch width layerer, and a longest path layerer by Nikolov et al. [33, 34], Coffman-Graham layer assignment [3], network-simplex layer assignment and node placement by Gansner et al. [20], Forster constraint resolving during barycenter crossing minimization [14], Sugiyama style crossing minimization [62], Brandes and Köpf node placement [1], linear segment node placement based on Sander [45], hyper-edge cycle detection based work of Eades et al. [12], orthogonal edge routing based on Sander and Di Battista et al. [4, 45], spline edge routing based on the approach of Sederberg [56], and a scan-line algorithm based on Lengauer [31].

3 ELK Features

In the following, we present how ELK Layered deals with compound graphs and model order as an example for two ELK features.

¹⁹ <https://www.eclipse.org/legal/ECA.php>

Compound Graphs and Hierarchical Ports Compound graphs or hierarchical graphs are laid out bottom-up by default. The inner graph of a compound node determines the position of hierarchical ports, which cannot be changed by the parent. In Fig. 3a the order of the inner graph inside **ChronoLogic**, specifically the order of the outputs of the inner reaction (gray arrow without a number), determines the order of the hierarchical ports **d**, **s**, and **m** on the **ChronoLogic** node. At the same time, the order of the reactions 1 - 3 (numbered gray arrows) inside **PrintOutput** determines the order of the hierarchical input ports **m**, **s**, and **d** of **PrintOutput**. If these ordering decisions are made independently, edge crossings, as the one in Fig. 3a, cannot be prevented. ELK Layered can solve this problem by considering the hierarchical edges and their destination in the child nodes if desired. This increases the complexity of the layout step and is invasive in the algorithm structure. Therefore, many strategies such as different layout directions in different hierarchies and all strategies that influence ordering do not work together with this option.



(a) The hierarchical edges may create preventable edge crossings. (b) Considering the edges across hierarchy levels prevents edge crossings.

Figure 3: An abbreviated **Chrono** Lingua Franca [32] program by Schulz-Rosengarten and von Hanxleden layouted bottom-up with ELK.

Model Order Model order represents the order of the input model, which can often be used for layout decisions, since “the input data can be expected to determine the choice ...” (see Jünger and Mutzel [29], p. 31). Domrös et al. describe how ELK Layered utilizes model order as a tie-breaker or a constraint not only during cycle breaking but also during layer assignment and crossing minimization to capture the user intention. This approach tends to create better layouts that can be influenced more directly by users and translates secondary notation from a (textual) input model into the layout. Furthermore, it solves common problems created by the bottom-up layout approach for compound graphs, as seen in Fig. 3. If one assumes that the user will always order ports, outputs, and inputs such that **m**, **s**, and **d** are consistently ordered, model order prevents the problem in Fig. 3a from occurring. Model order crossing minimization will create the crossing-free drawing depicted in Fig. 3b without using a complex hierarchy-aware layout algorithm.

4 Implementation

ELK consists of 146000 lines of handwritten Java code (nearly 60000 lines alone for ELK Layered), 3500 lines of algorithm metadata, which generate the layout options and algorithm documentations, and five Xtext DSLs. The Google Web Toolkit transpiles ELK-native algorithms into the JavaScript package `elkjs` to make ELK usable in web-applications. However, `elkjs` is not as performant for larger graphs as the original ELK²⁰.

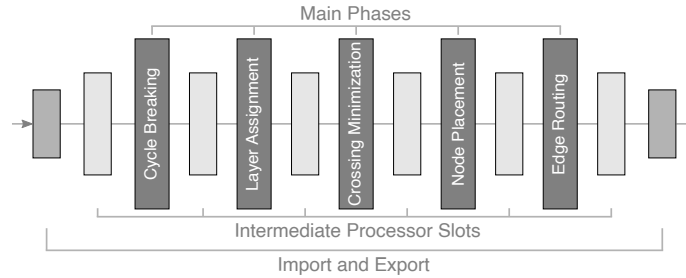


Figure 4: The five phases (dark gray) of ELK Layered with intermediate processor slots (light gray) before, between, and after each phase with an enclosing import and export step (gray) [53].

4.1 Algorithm Architecture

ELK provides a phase and processor infrastructure with import and export steps, as seen in Fig. 4. The structuring of ELK Layered into five instead of the traditional three phases makes it easier to interchange different layout strategies. The import step transforms a general ELK graph into a layered graph and transforms the layout problem into a left-to-right layout. The exporter reverses these changes after the layout. This allows algorithm-specific data-structures and abstracts from problems such as layout direction or implicit ports. Additionally, pre- and post-processing may be done in the six intermediate processing slots.

Before, between, and after each of the phases, each of the 57 intermediate processors (see Fig. 5) can be executed with the limitation that they need to have a static execution order [53]. The use of intermediate processors might slightly increase the running time of the algorithm. However, these processors make the algorithm much more maintainable by avoiding duplicate code and allowing to adopt algorithms for different sub-problems. This is done programmatically to keep the phase-processor dependencies maintainable. E.g., the `GREEDY` cycle breaking strategy can define that it needs the `REVERSED_EDGE_RESTORER` processor, which re-reverses the reversed edges, to be executed after phase five.

²⁰ <https://github.com/kieler/elk-speed>

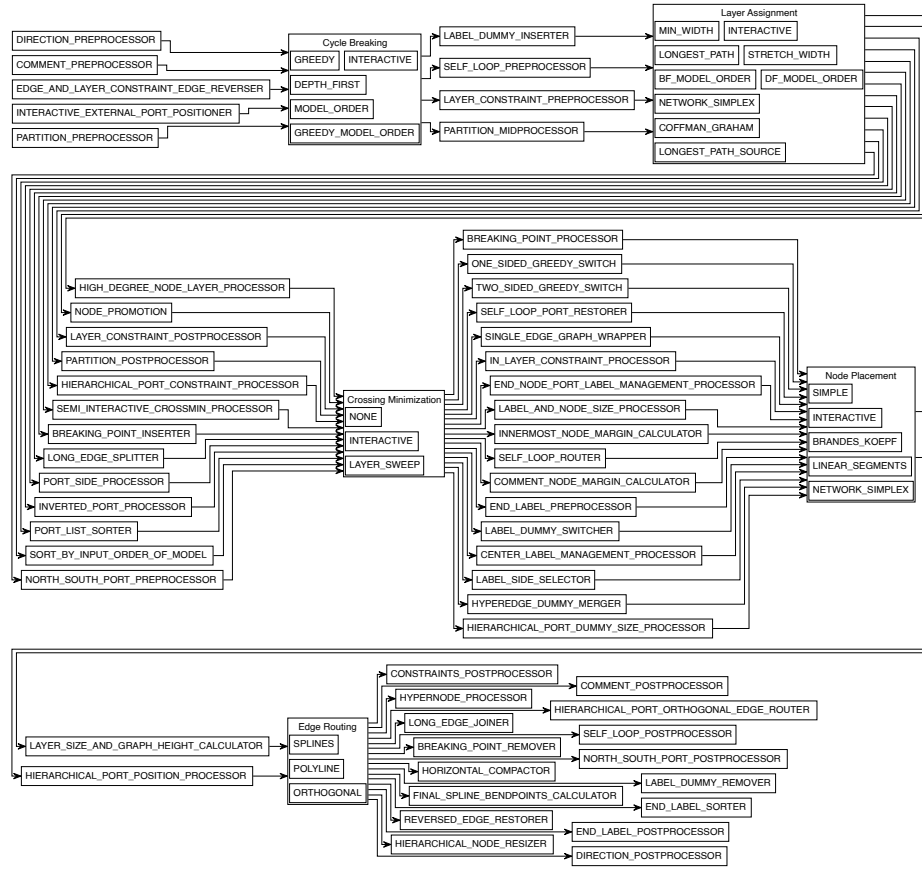


Figure 5: All ELK Layered processors, ordered by their execution order, and layout strategies, layouted with ELK Layered and styled with KLightD [46].

Since ELK has its roots in the Eclipse RCA KIELER, it additionally contributes debugging and logging views for Eclipse IDE. Furthermore, an Eclipse project wizard allows easy creation of algorithms, which is mainly used for teaching purposes.

4.2 Lessons Learned

In the following, we describe what users want from a layout library based on feature requests and questions from the ELK community. We reflect on the maintenance and documentation practices of ELK, and present features that should be removed or reworked to foster ELK’s future success.

Listening to Users We interpret the success of ELK (and other layout libraries) to indicate that the idea of automatic layout is slowly but surely gaining ground

in the practical usage of graphical languages. However, to be truly successful, one still has to listen to the users. Specifically, in our experience, users generally seem to be quite pleased with the layout results, at least as far as the aesthetic criteria are concerned that drawing algorithms are typically evaluated with, such as reducing edge crossings. However, users may get frustrated if they appear to have “little control” over actual results. When somebody’s mental map of a graph is inflicted because drawings change drastically after minimal changes of the graph, or, even worse, without any changes of the graph, this also causes irritations. For quite some time, we experimented with different approaches to give the users more control and stability. For example, we provide annotations to the graph that specify the nodes that should be placed in the same layer [35]. However, these did not really seem to catch on, probably for lack of awareness of these options.

We, therefore, lately focused on a different approach to give modelers control, and to enhance layout stability as well, which is the *model order* approach mentioned earlier. Basically, we consider it—in most cases—a mistake to consider a graph as a *set* of nodes and *set* of edges, as done in most of the literature. In our experience, it is much more practical to treat nodes and edges as ordered lists, with the order stemming e.g. from a textual input file. The model order should only be violated if the graph can clearly be improved by it. Whenever there are equally good alternative drawings available, one should use model order as a “tie-breaker”, i.e., choose the drawing that is closer to the model order [8].

For the same reasoning, while we originally used randomness rather liberally in our algorithms, in the hope to increase the chances of optimizing drawings according to some aesthetic criteria, we by now try to avoid randomness unless there are very strong reasons to include it.

Maintaining and Documenting the Project Since its beginnings, ELK typically had (only) one to three active developers. Furthermore, ELK is research driven, and whatever software engineering and maintenance it receives is mostly provided by motivated graduate students “on the side.” Software engineering tasks such as adding documentation, fixing bugs, testing, updating versions, interacting with users, and managing builds and releases cannot be the main focus of researchers, which mainly focus on new features and strategies for ELK for their dissertation. At the same time, in computer science it is not common to fund software engineers that operate and maintain open-source libraries such as ELK by the research institution itself. On the long run this is a severe impediment for research that builds on powerful, reliable open-source platforms. The situation is different in other fields. E.g., in experimental physics it is understood that a research institution not only needs scientists that actually conduct and publish research, but that there are also highly skilled lab engineers required that tend to the infrastructure. Given these circumstances, we originally would not have expected ELK’s longevity and popularity. E.g., elkjs has received over 1200 “stars” on GitHub by now.

This maintenance effort also enables research. Even though the first contributors to ELK have graduated long ago, ELK still provides a stable research infrastructure for complex layout algorithms. E.g., the recently added model order strategies require a highly modular layout algorithm with exchangeable layout strategies for the Sugiyama algorithm and a flexible system for pre- and post-processing, which is provided by the phase and processor infrastructure of ELK presented in Fig. 4.

As part of the maintenance effort, early ELK developers lived and enforced good coding conventions consisting of regular code reviews and refactoring as part of the release cycle. These made sure that the algorithm structure did not degrade over time. Most of the releases in Fig. 1 were accompanied by a refactoring process, especially for the initial release of KLayout Layered, ELK, and elkjs. E.g., as part of the creation of ELK the graph format was customized for layout and anything regarding visualization was removed. Moreover, the transpilation to elkjs required elimination of “hacks” such as Java reflection.

The move to the Eclipse Foundation was a time-consuming effort but finally helped to support the maintenance, documentation, and legal efforts. As a result, ELK has a stable release and build management system, which does not depend on sometimes unreliable university server infrastructure. Release reviews guarantee a certain code, documentation, and process quality, and the Eclipse Contributor Agreement prevents legal issues.

Additionally, documentation is partly enforced by convention and syntax. The metadata language that describes algorithms configurations enforces documentation, which is hosted on the Eclipse project web-page together with additional documentation in form of guides, examples, and blog posts. However, as in many academic projects, the end user documentation lacks behind. Interactive support channels proved necessary. As a result, ELK utilizes the online ELK editor `elklive` (see Sec. 2) and the ELK Gitter chat²¹, next to the GitHub issues to communicate with its user base.

Not only the end users but also developers need documentation. For the layered algorithm, processors are carefully ordered in each slot based on dependencies between, which corresponds to the processor ordering in Fig. 5. Each processor documentation lists its designated slots, pre- and post-conditions, and in-slot dependencies to other processors in form of Javadoc. However, the documentation of dependencies is not always done carefully since dependencies between processors and between different layout options get exponentially more complex with the number of features. Moreover, new features might require constraints that were previously deemed irrelevant. E.g., the model order strategies might reorder nodes and edges and assume that other processors will not introduce additional nodes and will respect this ordering. This was originally not considered since a graph was previously assumed to consist of *unordered* sets of nodes and edges.

²¹ <https://gitter.im/eclipse/elk>

Removing Outdated Features The metadata-compiler, which compiles the algorithm metadata into Java and generates algorithm and layout option documentation, on the one hand makes it easier for ELK developers to add new layout options. On the other hand it requires an installed metadata-compiler in the development environment, which is, hence, limited to the Eclipse IDE. In its current state, this limits collaboration in the open-source project. We experience that highly motivated users want to collaborate but fail to install the necessary tooling. At the same time, the metadata-compiler generates most of the algorithm documentation. Reworking this would require an effort but would greatly help ELK gain additional contributors.

Debugging tools, which proved to be essential for algorithm development, only exist for the Eclipse IDE. As part of future work the Eclipse-specific functions as well as part of the service architecture should move in a separate repository, leaving only the algorithms and their infrastructure in the ELK core project.

Refactoring should not only be used to improve code quality and to identify bottlenecks but also to remove features. Some features, such as the diploma thesis of Fuhrmann (see Fig. 1) about compound graph layout, were too complex to be maintained or do not fit into the algorithm phase structure. Such features may threaten the whole layout library since they increase the maintenance effort and facilitate the usage of “hacks” to build desired functionality. Hence, regular refactoring needs to identify such features and deprecate them to decrease unnecessary maintenance efforts.

5 Examples

In the following, we showcase how ELK is used to layout SCCharts [24], a synchronous language that models control-flow, and Lingua Franca [32], a polyglot coordination language that models data-flow. All the following models are styled with KLighD [46], since ELK does not support colors, fonts, and non-rectangular forms but only computes coordinates and sizes for rectangles and routes lines.

The Lingua Franca model in Fig. 6 showcases hierarchical edges using ELK Layered. The reactors **WaitingRoom** and **Customer** both have internal behavior, which is also layouted using ELK Layered. Here, model order orders the internal behavior of **WaitingRoom** and **Customer** as their numbering suggests. The inner graph, the ports, and the port labels determine the size of a compound node. ELK Layered is here configured to place the port labels consistently above the corresponding port. The placement of the vertical edge segments between **WaitingRoom** and **Customer** minimizes the used horizontal space. The port-label spacing and port-port spacing makes sure that the labels does not overlap with the arrow shaped ports and that no ports overlap.

The SCChart in Fig. 7 showcases how different layout algorithms can be used in the same model. Here, rectangle packings and layered layouts alternate. Moreover, directions of layered layouts also alternate, creating drawings with a better aspect ratio. The graph inside **ProcessInputs** is layouted top-to-bottom, and the graph inside **Running** and **GenClkState** is layouted left-to-right. Model order

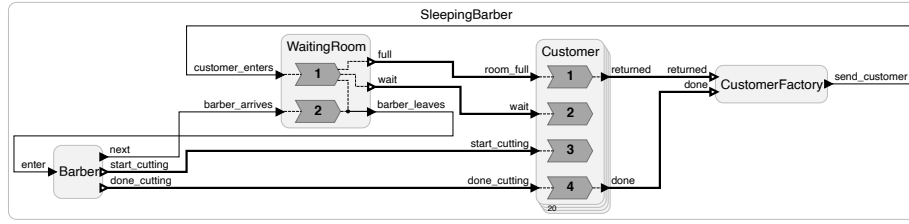


Figure 6: The sleeping barber Lingua Franca model taken from <https://github.com/lf-lang/examples-lingua-franca>.

is used as a tie-breaker, which makes sure that the outgoing and incoming edges of the initial **Pause** state are correctly ordered. Model order also highlights the control-flow loops, since the textual state ordering inside an SCChart usually determines the choice for a backward edge. Edge-label-management positions edge labels such that they have semantic line breaks. Moreover, edge labels are positioned on an edge such that labels can be clearly matched with the transitions they refer to. The “on-edge” positioning tends to reduce the drawing height. Padding makes sure that the inner layered graph will not overlap with the triangle in the upper left corner or the label next to it. In a corresponding IDE, the layout direction, label shortening strategies, and other layout options can be directly configured, e.g., to get a drawing to fit a certain aspect ratio or to create a desired layout.

The layout algorithm and all mentioned strategies are suitable in interactive scenarios and the layout time is for all shown models in millisecond range.

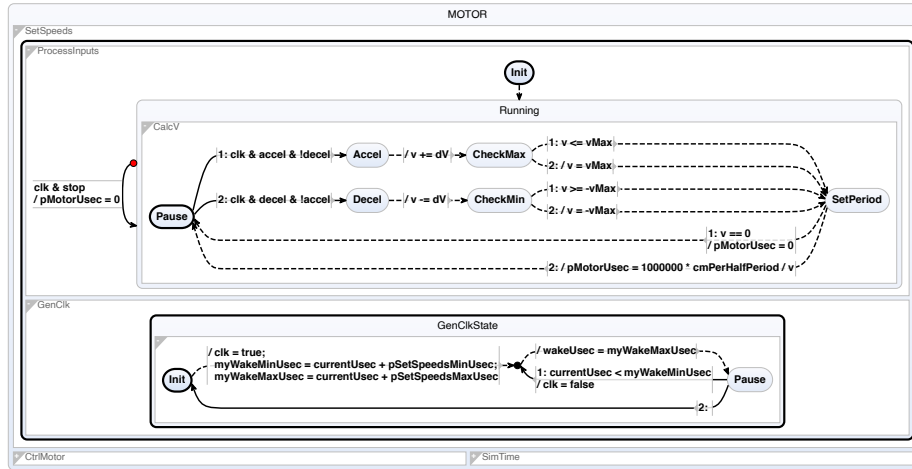


Figure 7: The motor SCChart.

6 Conclusion and Outlook

ELK and the algorithms behind it are used as the basis of several academic and commercial projects, while only being developed by a handful of people over the course of nearly fifteen years. Through its migration into the Eclipse Foundation and focusing not only on individual research but also on software engineering, it will hopefully live on for some time.

Graduate students not only spent a considerable amount of time engineering ELK but also implemented good coding and documentation practices to maintain a high code quality. Time spent interacting with the community through multiple interactive collaboration channels allows identifying highly requested features and developing practically relevant algorithms. These new use cases spawn new research topics, which again attracts future developers for ELK in form of graduate students. However, ELK could greatly benefit from a non-scientific software engineer.

Future work on ELK will include refactoring as well as general maintenance issues to make ELK more robust for future development. ELK will add support for top-down layout as an alternative to the presented bottom-up layout strategy for compound graphs, which can be used to add map-navigation-like tooling for complex graphs. Moreover, ELK Layered will get support for selective in-layer edges, which are usually forbidden in layered layouts. We also plan to add more model order configurations as well as model order support using different model order groups used to discern the ordering of different kind of graph elements.

7 Acknowledgments

We thank all the developers that put their heart and soul into ELK, which are partly mentioned in Fig. 1. We are indebted to the numerous users of ELK providing valuable feedback. In particular, Edward Lee and his group have been enthusiastic and supportive for many years now, which helped to considerably broaden the scope and utility of ELK.

References

1. Brandes, U., Köpf, B.: Fast and simple horizontal coordinate assignment. In: Mutzel, P., Jünger, M., Leipert, S. (eds.) *Proceedings of the 9th International Symposium on Graph Drawing (GD '01)*. LNCS, vol. 2265, pp. 33–36. Springer (2002). <https://doi.org/10.1007/3-540-45848-4>
2. Chimani, M., Gutwenger, C., Jünger, M., Klein, K., Mutzel, P., Schulz, M.: The Open Graph Drawing Framework. Poster at the 15th International Symposium on Graph Drawing (GD07) (2007)
3. Coffman, E.G., Graham, R.L.: Optimal scheduling for two-processor systems. *Acta Informatica* 1(3), 200–213 (1972). <https://doi.org/10.1007/BF00288685>
4. Di Battista, G., Eades, P., Tamassia, R., Tollis, I.G.: *Graph Drawing: Algorithms for the Visualization of Graphs*. Prentice Hall (1999)

5. Domrös, S., von Hanxleden, R.: Preserving order during crossing minimization in Sugiyama layouts. In: Proceedings of the 17th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2022) - Volume 3: IVAPP. pp. 156–163. INSTICC, SciTePress (2022). <https://doi.org/10.5220/0010833800003124>
6. Domrös, S., Lucas, D., von Hanxleden, R., Jansen, K.: On order-preserving, gap-avoiding rectangle packing. In: Proceedings of the 16th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2021) - Volume 3: IVAPP. pp. 38–49. INSTICC, SciTePress (2021). <https://doi.org/10.5220/0010186400380049>
7. Domrös, S., Lucas, D., von Hanxleden, R., Jansen, K.: Revisiting order-preserving, gap-avoiding rectangle packing. In: Computer Vision, Imaging and Computer Graphics Theory and Applications. pp. 183–205. Springer International Publishing, Cham (2023)
8. Domrös, S., Riepe, M., von Hanxleden, R.: Model order in Sugiyama layouts. In: Proceedings of the 18th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2023) - Volume 3: IVAPP. pp. 77–88. INSTICC, SciTePress (2023). <https://doi.org/10.5220/0011656700003417>
9. Dwyer, T., Koren, Y.: Dig-CoLa: directed graph layout through constrained energy minimization. In: Proceedings of the IEEE Symposium on Information Visualization (INFOVIS'05). pp. 65–72 (Oct 2005). <https://doi.org/10.1109/INFVIS.2005.1532130>
10. Dwyer, T., Marriott, K., Wybrow, M.: Dunnart: A constraint-based network diagram authoring tool. In: Revised Papers of the 16th International Symposium on Graph Drawing (GD '08). LNCS, vol. 5417, pp. 420–431. Springer (2009). <https://doi.org/10.1007/978-3-642-00219-9>
11. Eades, P.: A heuristic for graph drawing. *Congressus Numerantium* **42**, 149–160 (1984)
12. Eades, P., Lin, X., Smyth, W.F.: A fast and effective heuristic for the feedback arc set problem. *Information Processing Letters* **47**(6), 319–323 (1993). [https://doi.org/10.1016/0020-0190\(93\)90079-O](https://doi.org/10.1016/0020-0190(93)90079-O)
13. Fittkau, F., Krause, A., Hasselbring, W.: Software landscape and application visualization for system comprehension with explorviz. *Information and Software Technology* **87**, 259–277 (2017)
14. Forster, M.: A fast and simple heuristic for constrained two-level crossing reduction. In: Proceedings of the 12th International Symposium on Graph Drawing (GD '04), LNCS, vol. 3383, pp. 206–216. Springer (2005)
15. Franz, M., Lopes, C.T., Huck, G., Dong, Y., Sumer, O., Bader, G.D.: Cytoscape.js: A graph theory library for visualisation and analysis. *Bioinformatics* **32**(2), 309–311 (2016)
16. Frey, P., von Hanxleden, R., Krüger, C., Rüegg, U., Schneider, C., Spönemann, M.: Efficient exploration of complex data flow models. In: Proceedings of Modellierung 2014. pp. 321–336 (2014)
17. Fruchterman, T.M.J., Reingold, E.M.: Graph drawing by force-directed placement. *Software—Practice & Experience* **21**(11), 1129–1164 (1991). <https://doi.org/http://dx.doi.org/10.1002/spe.4380211102>
18. Fuhrmann, H., von Hanxleden, R.: Taming graphical modeling. In: Proceedings of the ACM/IEEE 13th International Conference on Model Driven Engineering Languages and Systems (MoDELS '10). LNCS, vol. 6394, pp. 196–210. Springer (Oct 2010). <https://doi.org/10.1007/978-3-642-16145-2>

19. Gansner, E.R., Koren, Y., North, S.C.: Graph drawing by stress majorization. In: Pach, J. (ed.) *Graph Drawing*. LNCS, vol. 3383. Springer Berlin Heidelberg (2005). https://doi.org/10.1007/978-3-540-31843-9_25
20. Gansner, E.R., Koutsofios, E., North, S.C., Vo, K.P.: A technique for drawing directed graphs. *Software Engineering* **19**(3), 214–230 (1993)
21. Gansner, E.R., North, S.C.: An open graph visualization system and its applications to software engineering. *Software—Practice and Experience* **30**(11), 1203–1234 (2000)
22. Glaser, P.L., Bork, D.: The bigER tool - hybrid textual and graphical modeling of entity relationships in VS Code. In: *2021 IEEE 25th International Enterprise Distributed Object Computing Workshop (EDOCW)*. pp. 337–340. IEEE (2021)
23. Gutwenger, C., von Hanxleden, R., Mutzel, P., Rüegg, U., Spönemann, M.: Examining the compactness of automatic layout algorithms for practical diagrams. In: *Proceedings of the Workshop on Graph Visualization in Practice (GraphViP '14)*. pp. 42–52 (2014)
24. von Hanxleden, R., Duderstadt, B., Motika, C., Smyth, S., Mendler, M., Aguado, J., Mercer, S., O'Brien, O.: SCCharts: Sequentially Constructive Statecharts for safety-critical applications. In: *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '14)*. pp. 372–383. ACM, Edinburgh, UK (Jun 2014). <https://doi.org/10.1145/2594291.2594310>
25. von Hanxleden, R., Fuhrmann, H., Spönemann, M.: KIELER—The KIEL Integrated Environment for Layout Eclipse Rich Client. In: *Proceedings of the Design, Automation and Test in Europe University Booth (DATE '11)*. Grenoble, France (Mar 2011)
26. von Hanxleden, R., Lee, E.A., Fuhrmann, H., Schulz-Rosengarten, A., Domrös, S., Lohstroh, M., Bateni, S., Menard, C.: Pragmatics twelve years later: A report on Lingua Franca. In: *11th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA)*. Lecture Notes in Computer Science, vol. 13702, pp. 60–89. Springer, Rhodes, Greece (Oct 2022). https://doi.org/10.1007/978-3-031-19756-7_5
27. Harel, D.: Statecharts: A visual formalism for complex systems. *Science of Computer Programming* **8**(3), 231–274 (Jun 1987)
28. Jabrayilov, A., Mallach, S., Mutzel, P., Rüegg, U., von Hanxleden, R.: Compact layered drawings of general directed graphs. In: *Proceedings of the 24th International Symposium on Graph Drawing and Network Visualization (GD '16)*. pp. 209–221 (2016). <https://doi.org/10.1007/978-3-319-50106-2>
29. Jünger, M., Mutzel, P. (eds.): *Graph Drawing Software*. Springer (2004)
30. Klauske, L.K., Schulze, C.D., Spönemann, M., von Hanxleden, R.: Improved layout for data flow diagrams with port constraints. In: *Proceedings of the 7th International Conference on the Theory and Application of Diagrams (DIAGRAMS '12)*. LNAI, vol. 7352, pp. 65–79. Springer (2012). <https://doi.org/10.1007/978-3-642-31223-6>
31. Lengauer, T.: *Combinatorial Algorithms for Integrated Circuit Layout*. John Wiley & Sons, Inc., New York, NY, USA (1990)
32. Lohstroh, M., Menard, C., Bateni, S., Lee, E.A.: Toward a Lingua Franca for deterministic concurrent systems. *ACM Transactions on Embedded Computing Systems (TECS)* **20**(4), Article 36 (May 2021). <https://doi.org/10.1145/3448128>
33. Nikolov, N.S., Tarassov, A.: Graph layering by promotion of nodes. *Discrete Applied Mathematics* **154**(5), 848–860 (2006). <https://doi.org/10.1016/j.dam.2005.05.023>

34. Nikolov, N.S., Tarassov, A., Branke, J.: In search for efficient heuristics for minimum-width graph layering with consideration of dummy nodes. *Journal of Experimental Algorithmics* **10** (2005). <https://doi.org/10.1145/1064546.1180618>
35. Petzold, J., Domrös, S., Schönberner, C., von Hanxleden, R.: An interactive graph layout constraint framework. In: *Proceedings of the 18th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2023) - Volume 3: IVAPP*, pp. 240–247. INSTICC, SciTePress (2023). <https://doi.org/10.5220/0011803000003417>, with accompanying poster
36. Prochnow, S., Traulsen, C.: KIEL—textual and graphical representations of state-charts. Presentation at the 12th Synchronous Workshop (SYNCHRON '05), Malta (Nov 2005)
37. Ptolemaeus, C. (ed.): *System Design, Modeling, and Simulation using Ptolemy II*. Ptolemy.org (2014), <http://ptolemy.org/books/Systems>
38. Rüegg, U.: Sugiyama layouts for prescribed drawing areas. No. 2018/1 in *Kiel Computer Science Series*, Department of Computer Science (2018), Dissertation, Faculty of Engineering, Christian-Albrechts-Universität zu Kiel
39. Rüegg, U., Ehlers, T., Spönemann, M., von Hanxleden, R.: A generalization of the directed graph layering problem. In: *Proceedings of the 24th International Symposium on Graph Drawing and Network Visualization (GD '16)*, pp. 196–208 (2016). https://doi.org/10.1007/978-3-319-50106-2_16
40. Rüegg, U., von Hanxleden, R.: Wrapping layered graphs. In: *Proceedings of the 10th International Conference on the Theory and Application of Diagrams (DIAGRAMS '18)*, pp. 743–747. Springer (2018)
41. Rüegg, U., Lakkundi, R., Prasad, A., Kodaganur, A., Schulze, C.D., von Hanxleden, R.: Incremental diagram layout for automated model migration. In: *Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems (MoDELS '16)*, pp. 185–195 (2016). <https://doi.org/10.1145/2976767.2976805>
42. Rüegg, U., Schulze, C.D., Carstens, J.J., von Hanxleden, R.: Size- and port-aware horizontal node coordinate assignment. In: *Proceedings of the 23rd International Symposium on Graph Drawing and Network Visualization (GD '15)*, pp. 139–150 (2015). https://doi.org/10.1007/978-3-319-27261-0_12
43. Rüegg, U., Schulze, C.D., Grevismühl, D., von Hanxleden, R.: Using one-dimensional compaction for smaller graph drawings. In: *Proceedings of the 9th International Conference on the Theory and Application of Diagrams (DIAGRAMS '16)*. LNCS, vol. 9781, pp. 212–218. Springer (2016). https://doi.org/10.1007/978-3-319-42333-3_16
44. Rüegg, U., Schulze, C.D., Sprung, C., Wechselberg, N., von Hanxleden, R.: Edge bundling for dataflow diagrams. Poster at the 24th International Symposium on Graph Drawing and Network Visualization (GD '16) (2016)
45. Sander, G.: A fast heuristic for hierarchical Manhattan layout. In: Brandenburg, F.J. (ed.) *Proceedings of the Symposium on Graph Drawing (GD '95)*. LNCS, vol. 1027, pp. 447–458. Springer (1996). <https://doi.org/10.1007/BFb0021828>
46. Schneider, C., Spönemann, M., von Hanxleden, R.: Just model! – Putting automatic synthesis of node-link-diagrams into practice. In: *Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC '13)*, pp. 75–82. IEEE, San Jose, CA, USA (Sep 2013). <https://doi.org/10.1109/VLHCC.2013.6645246>
47. Schulze, C.D.: Two opportunities and challenges of automatic layout in visual languages. In: *Proceedings of the ACM Student Research Competition at MODELS*

- 2016 co-located with the 19th International Conference on Model Driven Engineering Languages and Systems (MODELS 2016) (2016)
48. Schulze, C.D.: Text in Diagrams: Challenges to and Opportunities of Automatic Layout. No. 2019/4 in Kiel Computer Science Series, Department of Computer Science, CAU Kiel (2019). <https://doi.org/10.21941/kcss/2019/4>, dissertation, Faculty of Engineering, Kiel University.
 49. Schulze, C.D., von Hanxleden, R.: Automatic layout in the face of unattached comments. In: Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC '14). pp. 41–44. Melbourne, Australia (Jul 2014). <https://doi.org/10.1109/VLHCC.2014.6883019>
 50. Schulze, C.D., Hoops, G., von Hanxleden, R.: Automatic layout and label management for UML sequence diagrams. In: Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC '18) (2018)
 51. Schulze, C.D., Lasch, Y., von Hanxleden, R.: Label management: Keeping complex diagrams usable. In: Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC '16). pp. 3–11 (Sep 2016). <https://doi.org/10.1109/VLHCC.2016.7739657>
 52. Schulze, C.D., Plöger, C., von Hanxleden, R.: On comments in visual languages. In: Proceedings of the 9th International Conference on the Theory and Application of Diagrams (DIAGRAMS '16). pp. 219–225. LNCS, Springer (2016). https://doi.org/10.1007/978-3-319-42333-3_17
 53. Schulze, C.D., Spönemann, M., von Hanxleden, R.: Drawing layered graphs with port constraints. Journal of Visual Languages and Computing, Special Issue on Diagram Aesthetics and Layout **25**(2), 89–106 (2014). <https://doi.org/10.1016/j.jvlc.2013.11.005>
 54. Schulze, C.D., Spönemann, M., Schneider, C., von Hanxleden, R.: Two applications for transient views in software development environments (showpiece). In: Proceedings of the IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC '14). Melbourne, Australia (Jul 2014)
 55. Schulze, C.D., Wechselberg, N., von Hanxleden, R.: Edge label placement in layered graph drawing. In: Proceedings of the 10th International Conference on the Theory and Application of Diagrams (DIAGRAMS '18). pp. 71–78. LNCS, Springer (2018). https://doi.org/10.1007/978-3-319-91376-6_10
 56. Sederberg, T.W.: An introduction to b-spline curves. Brigham Young University (2005)
 57. Spönemann, M.: Graph layout support for model-driven engineering. No. 2015/2 in Kiel Computer Science Series, Department of Computer Science (2015), Dissertation, Faculty of Engineering, Christian-Albrechts-Universität zu Kiel
 58. Spönemann, M., Duderstadt, B., von Hanxleden, R.: Evolutionary meta layout of graphs. In: Proceedings of the 8th International Conference on the Theory and Application of Diagrams (DIAGRAMS '14). LNAI, vol. 8578, pp. 16–30. Springer (2014)
 59. Spönemann, M., Fuhrmann, H., von Hanxleden, R.: Automatic layout of data flow diagrams in KIELER and Ptolemy II. Technical Report 0914, Christian-Albrechts-Universität zu Kiel, Department of Computer Science (Jul 2009)
 60. Spönemann, M., Fuhrmann, H., von Hanxleden, R., Mutzel, P.: Port constraints in hierarchical layout of data flow diagrams. In: Proceedings of the 17th International Symposium on Graph Drawing (GD '09). LNCS, vol. 5849, pp. 135–146. Springer (2010). <https://doi.org/10.1007/978-3-642-11805-0>

61. Spönemann, M., Schulze, C.D., Rüegg, U., von Hanxleden, R.: Counting crossings for layered hypergraphs. In: Proceedings of the 8th International Conference on the Theory and Application of Diagrams (DIAGRAMS '14). LNAI, vol. 8578, pp. 9–15. Springer (2014). https://doi.org/10.1007/978-3-662-44043-8_2
62. Sugiyama, K., Tagawa, S., Toda, M.: Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man and Cybernetics* **11**(2), 109–125 (Feb 1981). <https://doi.org/10.1109/TSMC.1981.4308636>
63. Walker, II, J.Q.: A node-positioning algorithm for general trees. *Software – Practice and Experience* **20**(7), 685–705 (1990)
64. Wiese, R., Eiglsperger, M., Kaufmann, M.: yFiles: Visualization and automatic layout of graphs. In: Mutzel, P., Jünger, M., Leipert, S. (eds.) Proceedings of the 9th International Symposium on Graph Drawing (GD '01). LNCS, vol. 2265, pp. 588–590. Springer-Verlag (2001)
65. Wybrow, M., Marriott, K., Stuckey, P.J.: Orthogonal connector routing. In: Proceedings of the 17th International Symposium on Graph Drawing (GD '09). LNCS, vol. 5849, pp. 219–231. Springer (2010). <https://doi.org/10.1007/978-3-642-11805-0>
66. Zink, J., Walter, J., Baumeister, J., Wolff, A.: Layered drawing of undirected graphs with generalized port constraints. *Computational Geometry* **105–106**, 101886 (2022). <https://doi.org/https://doi.org/10.1016/j.comgeo.2022.101886>, <https://www.sciencedirect.com/science/article/pii/S0925772122000293>